

Reforma Traffic Counter v2

Detección y conteo de **Ecobici** · **Bicycle** · **Person** en Av. Reforma sobre Rubik Pi 3 (Snapdragon / Hexagon NPU) con modelo Edge Impulse `.eim`.

Estructura del workspace

```
reforma_counter/
|
├─ detector.py          ← script principal
├─ session_history.py   ← manejo de historial y snapshots por sesión
├─ requirements.txt
├─ setup.sh
|
├─ models/
|   └─ ecobicift_gnn_v2.eim    ← renombra el .eim aquí (sin espacios ni #)
|
├─ historial.json        ← se crea automáticamente al cerrar la sesión
|
├─ snapshots/
|   └─ sesion_001/          ← primera ejecución del programa
|       └─ snap_143022_123456.jpg
|           └─ snap_143022_123456.json
|       └─ sesion_002/
|           └─ ...
|       └─ ...              (máx 20 fotos por sesión; la más vieja se borra)
|
└─ logs/
    └─ detector.log
```

Setup — primera vez

```
# 1. Copia el workspace a la Rubik Pi
scp -r reforma_counter/ rubik@<IP>:~/
```

```
# 2. Entra al directorio
ssh rubik@<IP>
```

```
cd reforma_counter/

# 3. Renombra el modelo (evita problemas con # y espacios)
mkdir -p models
mv "ruta/al/ecobicift-linux-aarch64-qnn-v2-impulse #4.eim" \
    models/ecobicift_qnn_v2.eim
# Actualiza model_path en Config si lo renombraste diferente

# 4. Setup automático
chmod +x setup.sh
./setup.sh setup
```

Instalación sin internet (modo offline)

Descarga los wheels en tu PC antes de ir a campo:

```
# En tu PC con internet:
pip download edge_impulse_linux opencv-python-headless numpy \
    adafruit-circuitpython-ssd1306 adafruit-blinka pillow \
    -d wheels/

# Transfiere la carpeta wheels/ a la Rubik Pi
scp -r wheels/ rubik@<IP>:~/reforma_counter/

# En la Rubik Pi:
venv/bin/pip install --no-index --find-links=wheels/ -r requirements.txt
```

Ejecutar

```
./setup.sh run          # foreground (debug)
./setup.sh run-bg       # background (producción)
./setup.sh logs         # tail en vivo
./setup.sh status
./setup.sh stop
```

Tecla	Acción
Q	Salir y guardar sesión en historial
R	Reset de contadores e IDs (la sesión continúa)

Cómo acceder a la documentación generada

1. Historial de sesiones — `historial.json`

Se actualiza automáticamente al cerrar el programa con `q`. Contiene una entrada por cada ejecución.

Ver resumen legible en terminal:

```
cd reforma_counter/  
venv/bin/python session_history.py      # todas las sesiones  
venv/bin/python session_history.py last # solo la última
```

Salida de ejemplo:

```
HISTORIAL REFORMA COUNTER — 3 sesión(es)
```

```
Sesión #001 | 2024-05-12 09:00:00 → 2024-05-12 09:14:32  
Duración   : 00:14:32  
Personas   : 47  
Bicicletas : 12 (Ecobici: 5)  
Snapshots  : 8  en snapshots/sesion_001  
FPS prom.  : 14.3
```

```
TOTALES ACUMULADOS:  
Personas   : 134  
Bicicletas : 38 (Ecobici: 15)
```

Ver el JSON directamente:

```
cat historial.json | python3 -m json.tool # pretty-print  
# o si tienes jq instalado:  
jq '.' historial.json  
jq '[-1]' historial.json                # última sesión  
jq '[][.conteo.personas]' historial.json # solo el campo personas de cada sesión
```

Copiar historial a tu PC para análisis:

```
# Desde tu PC:
scp rubik@<IP>:~/reforma_counter/historial.json .
scp -r rubik@<IP>:~/reforma_counter/snapshots/ .
```

2. Snapshots — snapshots/sesion_NNN/

Cada sesión tiene su propia carpeta. Máximo 20 fotos; al llegar a 21 se elimina la más antigua automáticamente.

Cada snapshot genera dos archivos:

```
snap_143022_123456.jpg    ← imagen con bbox, ROI y HUD de contadores
snap_143022_123456.json   ← metadatos (timestamp, contadores en ese momento)
```

Ver qué hay en cada sesión:

```
ls snapshots/sesion_001/
# Cuenta cuántas fotos tiene cada sesión:
for d in snapshots/sesion_*/; do echo "$d: $(ls $d/*.jpg 2>/dev/null | wc -l) fot
```

3. Log del sistema — logs/detector.log

Registra cada evento: inicio de sesión, modelo cargado, snapshots, errores.

```
tail -f logs/detector.log          # en vivo
grep "Sesión" logs/detector.log    # solo eventos de sesión
grep "ERROR" logs/detector.log     # solo errores
grep "Snapshot" logs/detector.log  # cada vez que se guardó una foto
```

Estructura de historial.json

```
[
  {
    "sesion": 1,
    "modelo": "ecobicift_qnn_v2.eim",
    "inicio": "2024-05-12 09:00:00",
    "fin": "2024-05-12 09:14:32",
```

```

    "duracion": "00:14:32",
    "duracion_seg": 872.0,
    "conteo": {
        "personas": 47,
        "bicicletas": 12,
        "ecobicis": 5
    },
    "frames_totales": 12436,
    "fps_promedio": 14.3,
    "snapshots": 8,
    "carpeta_snaps": "snapshots/sesion_001"
},
{
    "sesion": 2,
    ...
}
]

```

Nota: Si el programa se termina abruptamente (Ctrl+C o corte de luz), la sesión queda marcada como `"fin": "en curso"` en el JSON. Los contadores parciales **no** se guardan en ese caso. Si quieres persistencia ante crashes, abre un issue o pídelo como mejora.

OLED SSD1306 128x64 — conexión I2C

Rubik Pi 3 GPIO → SSD1306

3.3V	(pin 1)	→	VCC
GND	(pin 6)	→	GND
SCL	(pin 5)	→	SCL
SDA	(pin 3)	→	SDA

```

# Verificar que el chip responde
sudo apt install i2c-tools
i2cdetect -y 1      # busca 0x3C
# Si no aparece, prueba:
i2cdetect -y 0
i2cdetect -y 4

```

Posibles Tracebacks

ImportError: No module named 'session_history'

Asegúrate de ejecutar desde dentro de `reforma_counter/`:

```
cd reforma_counter/  
./setup.sh run
```

OSError: No such file or directory — modelo

```
# El nombre del archivo tiene espacios y #, renómbra-lo:  
mv "models/ecobicift-linux-aarch64-qnn-v2-impulse #4.eim" models/ecobicift_qnn_v2  
# Y actualiza model_path en Config dentro de detector.py
```

KeyError: 'image_input_width'

El modelo no inicializó correctamente (probablemente falla QNN).

```
EI_LOG_LEVEL=debug venv/bin/python detector.py 2>&1 | head -80  
# Busca si dice "fallback to CPU" o errores de libQnn  
find /usr /opt -name "libQnnHtp*" 2>/dev/null
```

PermissionError /dev/video* o /dev/i2c-*

```
sudo usermod -aG video,i2c $USER  
newgrp video && newgrp i2c
```

Bounding boxes desplazadas

El modelo puede devolver coordenadas normalizadas [0..1]. Edita en `detector.py` la sección de escalado:

```
# Si los valores de bb["x"] están entre 0 y 1:  
bx = int(bb["x"] * actual_w) # sin dividir entre inp_w  
by = int(bb["y"] * actual_h)  
bw = int(bb["width"] * actual_w)  
bh = int(bb["height"] * actual_h)
```

Confirma con `print(bb)` qué rango tienen.

adafruit_blinka — Pin SCL not found

```
# Para Rubik Pi necesitas el backend genérico:  
export BLINKA_MCP2221=1  
# O usa smbus2 directamente (pide implementación alternativa si lo necesitas)
```